Self-driving

Model Based

Model-free

environment :

not an MDP

highway

A : continuous

$a = \begin{pmatrix} a \\ \delta \end{pmatrix}$ — vehicle acceleration, steering angle

ranges?

(other options are available, incl. meta-actions : discrete lane & velocity)

O/S : different possible settings, incl. Lidar

↳ setting Kinematics : (simplest)

$O = \begin{pmatrix} \\ \end{pmatrix}$ (F across, V down)

vehicles:
self → ego-vehicle
others → vehicle 1
vehicle 2
⋮

what order? no specific order stated but maybe by lateral distance closest

R : $R(s,a) = a\left(\frac{v - v_{min}}{v_{max} - v_{min}}\right) - b \text{ collision}$

v_{min}, v_{max} & current v of vehicle

↳ rewards fast driving

penalties can be configured in env config

a & b are coefficients to scale priorities

Vehicle models: • all dynamics obey a kinematic model and update

neighbour vehicles are Non-Markovian : change lane after certain random times

This is the key modelling problem

+ also non-Markovian by the fact that new vehicles appear in env unpredictably

f_{policy} , f_{env} , $T_{episode}$

$\text{policy\_steps} = T_{episode} \cdot f_{policy}$

$f_{env} \geq f_{policy}$

∴ for gamma :

how to redefine states :

parking

for algorithm demonstration in simpler contexts:

Parking :

ψ: radians

$s = (x, y, v_x, v_y, \cos\psi, \sin\psi)$

goal: → $s_g = (x, y, v_x, v_y, \cos\psi, \sin\psi)$

$R(s,a) = -\|s - s_g\|_{W,p}^{p} - b \text{ collision}$

where $\|x\|_{W,p} = \left(\sum_i |W_i x_i|^p\right)^{\frac{1}{p}}$ ← p norm: ~~'narrower peak of reward at the goal compared to Euclidean norm'~~

by standard in documentation : p = 0.5

W allows different terminal states to be penalised differently

~~actually~~ an MDP : RL methods should work better at the basis

Model-free Agent :

- using SB3
- the highway env, examples of agents & working parameters included
  ↳ DQN
  , CEM (on exact dynamics model)

would start by using the parameters & info I

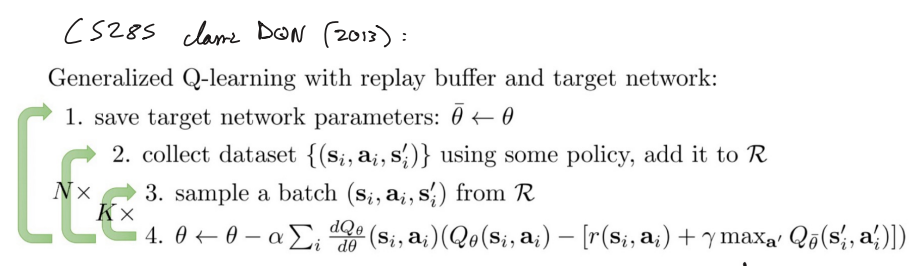
↳ SB3 DQN: • discrete A only (enforced by SB3, not DQN necessarily)

↳ in highway env: a custom quantisation of continuous actions (a, δ)

how fine? What range?

Could extend to dueling DDQN?

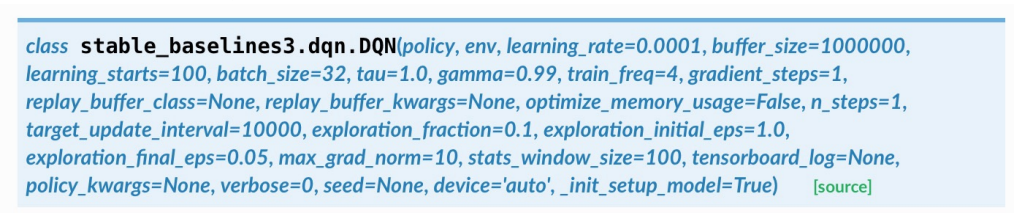
CS285 classic DQN (2013) :



Generalized Q-learning with replay buffer and target network:

```
N×  1. save target network parameters: $\bar{\theta} \leftarrow \theta$
    2. collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)\}$ using some policy, add it to $\mathcal{R}$
    K×  3. sample a batch $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$ from $\mathcal{R}$
        4. $\theta \leftarrow \theta - \alpha \sum_i \frac{dQ_\theta}{d\theta}(\mathbf{s}_i, \mathbf{a}_i)(Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - [r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_{\bar{\theta}}(\mathbf{s}'_i, \mathbf{a}'_i)])$
```

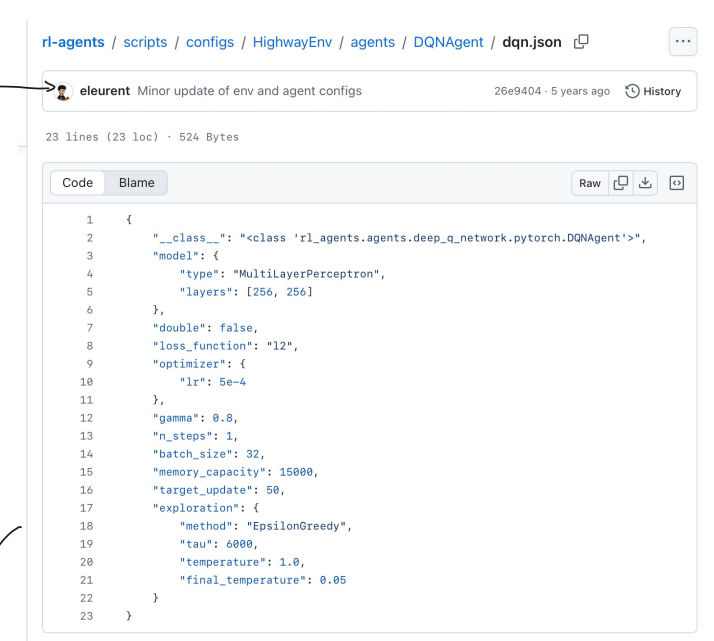
in SB3 we have the following parameters : (all parameters explained in SB3 documentation)



class **stable_baselines3.dqn.DQN**(*policy, env, learning_rate=0.0001, buffer_size=1000000, learning_starts=100, batch_size=32, tau=1.0, gamma=0.99, train_freq=4, gradient_steps=1, replay_buffer_class=None, replay_buffer_kwargs=None, optimize_memory_usage=False, n_steps=1, target_update_interval=10000, exploration_fraction=0.1, exploration_initial_eps=1.0, exploration_final_eps=0.05, max_grad_norm=10, stats_window_size=100, tensorboard_log=None, policy_kwargs=None, verbose=0, seed=None, device='auto', _init_setup_model=True*) [source]

parameters found in highway env documentation

built highway env →



rl-agents / scripts / configs / HighwayEnv / agents / DQNAgent / dqn.json

eleurent Minor update of env and agent configs 26e9404 · 5 years ago History

23 lines (23 loc) · 524 Bytes

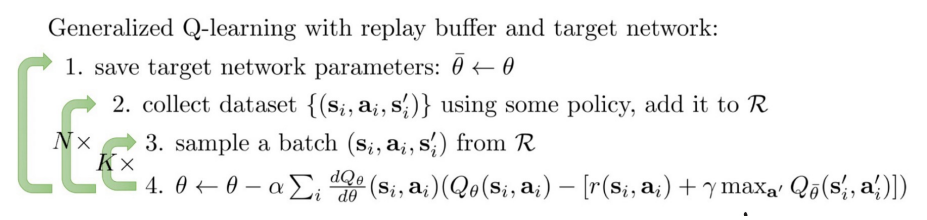
```
{
    "__class__": "<class 'rl_agents.agents.deep_q_network.pytorch.DQNAgent'>",
    "model": {
        "type": "MultiLayerPerceptron",
        "layers": [256, 256]
    },
    "double": false,
    "loss_function": "l2",
    "optimizer": {
        "lr": 5e-4
    },
    "gamma": 0.8,
    "n_steps": 1,
    "batch_size": 32,
    "memory_capacity": 15000,
    "target_update": 50,
    "exploration": {
        "method": "EpsilonGreedy",
        "tau": 6000,
        "temperature": 1.0,
        "final_temperature": 0.05
    }
}
```

↳ changed & 'working' in dqn-highway-3.zip

key change: DiscreteMetaAction allow to stop model slowing by driving off road

parking env :

DQN:



Generalized Q-learning with replay buffer and target network:

```
N×  1. save target network parameters: $\bar{\theta} \leftarrow \theta$
    2. collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)\}$ using some policy, add it to $\mathcal{R}$
    K×  3. sample a batch $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$ from $\mathcal{R}$
        4. $\theta \leftarrow \theta - \alpha \sum_i \frac{dQ_\theta}{d\theta}(\mathbf{s}_i, \mathbf{a}_i)(Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - [r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_{\bar{\theta}}(\mathbf{s}'_i, \mathbf{a}'_i)])$
```

how to choose Q_θ NN architecture? depth & width?

tricks to make work well:

• fix overestimation ? → double Q learning?

~~Look at~~ HER ?

dqn_horizon_4 : 'working' with no obstacles, 80,000 training steps

↳ just need to train for longer

DDQN : $y_i = r(s_i, a_i) + \gamma Q_{\bar{\theta}}\left(s'_i, \arg\max_a Q_\theta(s, a)\right)$

de-correlate noise :

↳ learns more 'pessimism' : learn around presence of cars better?

↳ still struggles to avoid cars : env full of cars runs very slowly / not (plateaus)

to make learning more efficient: HER?

Hindsight Experience Replay : technique that can be bolted onto any off-policy RL algorithm

useful in particular for sparse rewards (mean fail) : maybe good for parking?
& for multi-goal RL

⟹ from any rollout $(s_1, \ldots, s_T)$: reframes this as a successful rollout aiming to get to s_T

∴ learns how to drive to full set of areas : more information retained

(see paper online, 2017)

Speed up

Change model for parked cars

↳ ~~in alt env not~~ ~~fixed yet~~ FIX

Action Space

standard discretisation : actions per axis = 3 too low

↳ 5

$a = \begin{pmatrix} a \\ \delta \end{pmatrix}$ → defaults: acceleration $\in [-5, 5]$ ms^{-2}

steering $\in \left[-\frac{\pi}{4}, \frac{\pi}{4}\right]$ rad

Path length vs Crash penalty with γ

for long paths, actually greater reward to crash!

supposedly (Claude) : to set a good crash cost:

crash cost > $\frac{\text{max per step cost}}{1-\gamma}$?

$R = -\sum^{H} \gamma^i r_i$

suppose most expensive path: $r_i = r_{max}$

$H \to \infty$

$R = \frac{r_{max\,step}}{1-\gamma}$

this has to look more favourable than crashing

$|r_{crash}| > \left|\frac{r_{max\,step}}{1-\gamma}\right|$

$r_{max\,step}$: $R(s,a) = -\|s - s_g\|_{W,p}^{p} - b \text{ collision}$

where $\|x\|_{W,p} = \left(\sum_i |W_i x_i|^p\right)^{\frac{1}{p}}$

p = 0.5

?

with normalise, relative to what?

starts at -5 → -15 ?

γ 0.99 → 0.98

from run:
R crash = −10
γ = 0.98

learns to crash into wall

works but $r_{per\,step}$, avg ≈ −0.3 → −0.6

using −0.6 :

$|r_{crash}| > \frac{0.6}{1-\gamma}$

γ: 0.95

$|r_{crash}| > \boxed{12}$

new run: $\boxed{\begin{matrix} \gamma = 0.95 \\ R_{crash} = -20 \end{matrix}}$

Still fails

learning is unstable & plateaus / random oscillates

↳ likely cause: implicit multigoal learning via objective but $Q_\theta(s,a)$ isn't explicit about it

↳ gets confused ⟹ HER

scope split (not enough compute / time to run this many DQN tests)

multi-goal empty parking lot :

1e6 steps: works but learning crashes step ~550 000

WH-1

fine checkpoints

single goal full parking lot